CheckoutSentinel — Stage I Proposal Draft

ASUS UGen AI League · Battlefield Thunderstorm · Global Online Track

> Draft for the ≤20-page proposal deck + GitHub code-documentation link.
>
> Stage I deadline: **Oct 14, 2026, 17:00 (Asia/Taipei)**. Status: DRAFT — convert to deck PDF before submission.

Page 1 — Cover

**CheckoutSentinel: a real-time agentic guard for e-commerce checkout —

fraud judgment and cart rescue, decided in milliseconds, on local AI hardware.**

- Battlefield: **Thunderstorm** (high-performance development)
- Theme: **Workplace AI** (merchant operations · revenue protection)
- Track: Global Online · Solo entry
- Author: Arin Dewangan (Software Engineer, Bengaluru, India)
- GitHub: `https://github.com/arindewangan/checkoutsentinel` *(placeholder — repo published at submission)*
- Demo video (≤3 min): *(YouTube unlisted link — recorded at submission)*

Page 2 — Scenario: the checkout dilemma

Picture a mid-size D2C brand on a sale weekend: 40,000 checkouts a day. Their fraud filter — a rules engine bought in 2021 — does two things badly. It waves through a coordinated card-testing attack at 2 AM (■18 lakh in chargebacks by Monday), and it hard-declines 312 legitimate customers whose only crime was buying from a new phone while traveling. Meanwhile, 28,000 carts are abandoned; the marketing team's "10% off if you come back" email converts 4%.

This is the industry's normal. It doesn't have to be. Every one of those decisions — approve, challenge, rescue, hold — is a judgment call a good fraud analyst could make in ten seconds *if they could see every signal at once*.

CheckoutSentinel is that analyst, operating at machine speed: an agentic loop that scores, decides, acts, and explains — per transaction, in milliseconds, on the merchant's own hardware.

Page 3 — Problem description

Three compounding losses, one root cause (no judgment at checkout speed):

1. **Direct fraud loss.** Card-not-present fraud, account takeover, and triangulation fraud cost merchants an estimated tens of billions of dollars per year globally; the true cost per fraud dollar (chargebacks, fees, lost goods, manual review labor) is widely estimated at 3–4× the transaction value.
1. **False declines — the bigger bleed.** Over-tuned filters reject good customers; industry research consistently finds false-decline losses exceed actual fraud losses severalfold, and a declined shopper rarely returns.
1. **Abandoned carts.** ~70% of carts never convert. A meaningful fraction are price-sensitive-but-willing buyers; today's recovery tooling (static popups, batch emails) is dumb, slow, and trains customers to game discounts.

Existing tooling splits the problem across vendors — a risk score here, a 3DS orchestrator there, a Klaviyo flow for abandonment — with no unified decision-maker and no explanation layer a merchant can audit or trust.

Page 4 — Solution overview: CheckoutSentinel

CheckoutSentinel collapses fraud defense, step-up orchestration, and cart recovery into **one agentic loop** that runs on every checkout event:

1 **Sense** — a signal engine extracts fraud and intent features (amount anomaly, velocity, device/geo consistency, CVV retries, coupon behavior, email reputation) into an explainable 0–100 risk score.

1 **Decide** — an intervention policy takes the right-sized autonomous action: auto-approve (LOW), approve-with-async-flag (MEDIUM), step-up auth with a rescue incentive for good customers (HIGH), hold capture + P1 review (CRITICAL), or an autonomous win-back offer for abandoned carts.

1 **Explain** — a ~24B-class LLM agent writes a per-transaction analyst note: what it saw, why the action is right, and the conversion-vs-risk tradeoff.

Every decision is immutable and auditable.

The Stage I prototype in this repo implements the full loop end-to-end (simulated traffic, heuristic scorer, policy engine, labeled template narration, live dashboard). Stage II swaps in the real LLM agent on the B70 — the integration seam is already built.

Page 5 — Innovation & creativity

- **Agentic, not scored.** Incumbents return a number and leave the decision to the merchant. CheckoutSentinel *acts*: it challenges, rescues, or holds — and writes the analyst note justifying it. The agent is the product.
- **Conversion-aware fraud control.** The policy explicitly prices the conversion-vs-risk tradeoff: MEDIUM risk is approved (not blocked) with an async flag; HIGH risk on a *returning* shopper triggers verification *plus* an incentive. Fraud tools minimize fraud; we maximize *net revenue*.
- **Abandonment as a first-class event.** Rescue is part of the same loop, not a separate marketing tool — the agent that scored the cart decides the win-back.
- **Local-first agent inference.** Payment data never leaves the merchant's hardware — a structural advantage in regulated markets (PCI DSS, GDPR, India's DPDP Act) that cloud-API competitors cannot match.
- **Explainability by construction.** Every score decomposes into named, weighted signals; every action carries its reasons. Auditors and merchants see *why*, not just *what*.

Page 6 — Intended tools & methods (1/2): software

- **Application:** Python 3.12, Flask (demo service + dashboard), SQLite (demo) → Postgres + Redis Streams (production event backbone).
- **Agent runtime:** OpenAI-compatible inference server — llama.cpp (GGUF) or vLLM with Intel XPU backend (via oneAPI / Intel Extension for PyTorch). The prototype's `LLMClient` already speaks this protocol.
- **Agent framework:** a lean custom ReAct-style loop (sense → decide → narrate → log) in Stage I; evaluation of LangGraph-style orchestration in Stage II if multi-step tool use (e.g., querying the order history service) proves valuable.
- **Risk layer:** heuristic signal engine (Stage I, fully explainable) → calibrated gradient-boosted scorer + LLM-as-judge review in Stage II; the agent consumes signal attributions either way.
- **Integration:** store webhook + JS snippet; Shopify/WooCommerce plugin adapters on the commercial roadmap.
- **DevOps:** GitHub, pytest (26 tests, CI-ready), Docker; demo deploys as a single container.

Page 7 — Intended tools & methods (2/2): model strategy

The production agent is an **INT8-quantized ~24B-class open-weights LLM** (Apache 2.0-licensed for commercial use):

Candidate	License	INT8 footprint	Role
Mistral Small 4 (24B dense)	Apache 2.0	~24–26 GB Q8 / ~14 GB Q4_K_M	Primary — tool use, instruction following, 128K ctx
Qwen3.5-27B dense	Apache 2.0	~27 GB Q8 / ~17 GB Q4_K_M	Alternate — reasoning, multilingual
Gemma 4 31B dense	Apache 2.0	~18 GB Q4_K_M	Fallback at Q4

Why 24B-class in INT8: 24B × 1 byte ≈ 24 GB weights + 2–4 GB KV cache + overhead fits the B70's 32 GB ECC VRAM with margin; Q4_K_M (~14–15 GB) leaves room for long context and concurrent streams. INT8 is the B70's native strength (256 XMX engines, up to 367 INT8 TOPS).

Method: prompt-engineered analyst persona with structured function-style outputs (decision, reasons, narration); few-shot calibration on a labeled checkout corpus assembled in Stage II; a distilled <1B classifier head pre-filters obvious cases on CPU so the 24B agent spends tokens only on ambiguous, high-value decisions — the throughput engineering that makes per-transaction agents economical.

Page 8 — Hardware/software architecture

```
Store checkout service ■■webhook/stream■■■ CheckoutSentinel (ASUS UGen node)
■
■■■
```

Page 9 — Agent loop design (detail)

Per checkout event, the loop executes:

1. **Observe.** Assemble the transaction feature vector + customer history (rolling averages, velocity windows, device registry) from the store.
1. **Score.** Signal engine → 0–100 score, tier, and per-signal attributions (e.g., `geo_mismatch +15`, `cvv_retries +18`).
1. **Policy gate.** Deterministic table maps (tier, event, customer context) → intervention. The agent may *propose* an override in Stage II; overrides are logged with the same audit fields and capped by a merchant-configurable risk ceiling.
1. **Narrate.** The LLM agent receives (signals, tier, action) and writes the analyst note: observation → reasoning → action justification → conversion/risk tradeoff. Structured output keeps it machine-parseable.
1. **Act + log.** The intervention executes (API call to gateway/review queue/ messaging) and the full tuple (features, score, action, narration, latency) is written immutably.

Latency budget (production target): observe+score < 5 ms (CPU) → policy < 1 ms → agent narration p95 < 200 ms (INT8 24B on B70, short generation) → act+log < 10 ms. **End-to-end p95 < 300 ms** — inside the checkout's tolerance, where every 100 ms of added latency measurably costs conversion.

Page 10 — Why Thunderstorm hardware wins

Requirement	Cloud LLM API	CheckoutSentinel on B70
p95 decision latency	800 ms–2 s+ (network)	< 300 ms (local XMX)
Data residency	payment data leaves merchant	never leaves the box
Marginal cost/decision	per-token, forever	~$0 after hardware
Throughput scaling	rate limits, bills scale linearly	batching on 367 INT8 TOPS
Offline/edge deployment	impossible	single workstation/GPU node

The B70's 32 GB VRAM is the enabling constraint: it holds a 24B INT8 agent *plus* KV cache for concurrent streams on one card — no model sharding, no multi-GPU orchestration for the core product. 608 GB/s bandwidth feeds the XMX engines fast enough to keep time-to-first-token low, which is what makes the agent inline-capable rather than a batch afterthought.

Page 11 — Stage I demo plan (hands-on video, ≤3 min)

The ≤3-minute hands-on demo video walks through the working prototype:

1. **(0:00–0:30)** The problem in one breath + the live dashboard.
2. **(0:30–1:30)** Live simulated checkout stream: watch LOW/MEDIUM/HIGH/CRITICAL transactions arrive; the agent narrates each decision; the intervention log fills (step-up auths, a held capture, a rescued cart).
   On-screen labels mark simulated traffic and template narration throughout.
1. **(1:30–2:15)** Force an abandoned cart → rescue offer fires with a (simulated) discount code; force a fraud-flavored attempt (geo mismatch + CVV retries + amount spike) → capture held, P1 review.
1. **(2:15–3:00)** Architecture diagram + the B70/INT8 story: what changes in Stage II (real 24B agent, same protocol, same dashboard).
   No real payment data, no real discount codes, no fake AI claims — every simulated element is labeled on screen.

Page 12 — Expected outcomes (Stage II deliverables)

By the Dec 4 final submission:

1. **Working system** on the Thunderstorm VM/SDV environment: the full agent loop with a real INT8 24B-class agent served from an OpenAI-compatible endpoint, driven by a live demo storefront (Shopify sandbox or equivalent).
1. **Measured results:** p50/p95 decision latency, throughput (decisions/sec), VRAM footprint, and agent-narration quality scored against a labeled checkout corpus (precision/recall of the risk tiers, analyst-rated narration usefulness).
1. **Comparative note:** heuristic-only vs agent-augmented decisions on the same corpus — quantifying what the LLM adds (fewer false declines, better rescue targeting).
1. **Deployment artifact:** Docker Compose stack + setup guide so judges can reproduce the demo against the B70 profile.
1. **Commercial one-pager:** pricing model, integration path, and pilot offer for D2C merchants.

Page 13 — Commercial potential & feasibility

- **Market:** fraud-prevention + conversion-optimization tooling is a multi-billion-dollar market; CheckoutSentinel collapses three vendor categories (risk scoring, 3DS orchestration, cart recovery) into one agent.
- **Pricing:** per-decision SaaS ($0.01–0.03) or performance-based (share of recovered revenue). At 1M checkouts/month ≈ $10–30K/month — against fraud + false-decline + abandonment losses an order of magnitude larger.
- **Feasibility:** the demo proves the loop; the risk is model serving on Intel XPU (mitigated: llama.cpp/vLLM + IPEX paths are documented, and the CPU policy layer guarantees a working product even if the agent degrades).
- **Moat:** agentic intervention + narration is hard to replicate with rules; the local-inference story wins regulated verticals (EU, India DPDP) that cloud-only vendors serve poorly.
- **GTM:** Shopify/WooCommerce app stores → mid-market D2C → enterprise via the compliance story; design-partner pilots in Stage II.

Page 14 — Team

Solo entry — **Arin Dewangan**, Software Engineer (3 years), Bengaluru, India.

- End-to-end builder: agentic systems, full-stack web, ML-flavored backends, demo engineering (prior hackathon portfolio: agentic commerce, multi-agent code review, fraud-adjacent tooling).
- The solo scope is deliberate: one decision-maker, one coherent architecture, no coordination overhead against a Dec 4 deadline.

Page 15 — Timeline (Stage II: Nov 6 → Dec 4)

Week	Focus
Nov 6–12	Thunderstorm VM/SDV onboarding; inference server bring-up (llama.cpp/vLLM + Intel XPU); 24B INT8 candidate eval
Nov 13–19	Agent prompt + structured-output design; labeled checkout corpus v1; wire real agent into the loop
Nov 20–26	Latency/throughput tuning (batching, pre-filter); Shopify sandbox integration; dashboard hardening
Nov 27–Dec 3	Measurement (latency, precision/recall, narration quality); ≤3-min final demo video; deck finalization
Dec 4	Final submission

Page 16 — Risks & mitigations

Risk	Mitigation
Intel XPU inference-server friction	Two serving paths (llama.cpp GGUF, vLLM+IPEX); CPU policy layer keeps product functional regardless
24B INT8 quality on nuanced cases	Distilled pre-filter handles the obvious; agent only sees ambiguous cases; few-shot calibration corpus
Agent latency misses 300 ms p95	Short-form structured outputs, continuous batching, Q4_K_M fallback with larger KV headroom
Labeled fraud data scarcity	Synthetic corpus seeded from known fraud patterns + public datasets (e.g., IEEE-CIS style features), analyst-reviewed
Scope creep (solo)	Ruthless MVP: one storefront adapter, one agent, one dashboard — everything else is roadmap

Page 17 — References

1 Intel — Intel Arc Pro B70 product specifications (32 Xe2 cores, 256 XMX engines, up to 367 INT8 TOPS, 32 GB GDDR6 ECC, 608 GB/s, PCIe 5.0 x16). https://www.intel.com/content/www/us/en/products/sku/245797/intel-arc-pro-b70-graphics/specifications.html

1 ASUS UGen AI League — official contest page (contest.bhuntr.com) and circuitdigest.com event brief: $20,000+ prizes, Battlefield Thunderstorm (high-performance development), Global Online Track judged remotely, Stage I deadline Oct 14, 2026 17:00 (Asia/Taipei).

1 Mistral AI — Mistral Small 3.x model cards (24B dense, Apache 2.0, 128K context).

1 Qwen team — Qwen3.5 / Qwen3.8 model cards (Apache 2.0).

2 Google — Gemma 3 / Gemma 4 model cards (Apache 2.0).

3 IEEE Computational Intelligence Society — IEEE-CIS Fraud Detection dataset (feature taxonomy reference for the Stage II corpus).

1 Baymard Institute — cart abandonment rate research (~70% benchmark).

Page 18 — Appendix

Originality statement (contest rules §9(1)). CheckoutSentinel was conceived, named, and built fresh for the ASUS UGen AI League in October 2026. It has not been previously published, entered in any competition, or received any award. Architecture patterns are informed by the author's prior portfolio work; all code, copy, branding, and the concept itself are new and original to this entry.

IP & tax awareness (§9(2)). The author has read the contest rules, including the perpetual license grant to ASUS/Intel/Hailo for winning works and the 20% withholding tax for non-Taiwan residents, and enters with full awareness of these terms.

GitHub repository (code documentation link for the deck): `https://github.com/arindewangan/checkoutsentinel` *(placeholder — published with the full prototype, README architecture docs, model information, and 26-test suite at submission time)*

Judging-criteria mapping: Innovation/Creativity 30% → Pages 5, 9 (agentic intervention, conversion-aware policy, local-first inference); Commercial Potential/Feasibility 35% → Pages 8, 10, 13, 16 (architecture, hardware economics, pricing, de-risked plan); Practical Value 25% → Pages 4, 11, 12 (working prototype, measurable outcomes); Storytelling 10% → Pages 2, 11 (scenario narrative, structured demo video).